

자료구조 실습 보고서

[제 02 주] 동전가방

제출일 : 2018 년 3 월 25 일

201702081 최재범

1. 프로그램 설명서

a. 주요 알고리즘

- `public boolean remove(E element_p)` : 배열의 원소를 제거한 후 정리

배열에서 요소를 하나 제거한 후에는 그 뒤의 원소들의 인덱스를 하나씩 앞으로 옮기는 과정을 거쳐야 중간이 비지 않는 깔끔한 구조가 된다.

이를 구현하기 위하여 for 문으로 인덱스 순회를 하는데, 현재 인덱스의 요소에 다음 인덱스의 요소를 대입한다. 하지만 이렇게 순회를 계속하다가 마지막 인덱스까지 가버리면 다음 인덱스라는 것이 존재하지 않기 때문에 오류가 발생하게 된다.

따라서 순회하는 최대 인덱스를 마지막 요소의 이전 인덱스까지로 하며, 마지막 요소는 순회되지 않으므로 같은지 여부를 직접 확인시킨다.

마지막 요소는 다음 요소가 없으므로 `null` 을 넣어준다.

이렇게 하면 배열의 마지막 부분은 해결이 되지만, 다른 문제가 발생하는데, 만약 배열에 원소가 1 개만 들어있다면, for 문의 카운터 변수의 초기값이 0 이며 조건식이 0 미만인 경우로 설정되기 때문에 for 문 안의 내용이 실행되지 않게 된다.

따라서 if 문으로 배열의 사이즈가 1 일 때의 예외 상황을 만들어준다. 1 개만 들어있으면 뒤의 인덱스에서 옮길 것이 없으므로 그냥 `null` 을 넣어주기만 하면 된다.

b. 자료구조

이 프로그램은 전체적인 제어를 담당하며 상황에 맞게 메소드에게 명령을 내리는 `AppController` 클래스,

입력받아 반환하는 메소드와 출력을 담당하는 메소드를 포함하는 `AppView` 클래스,

코인 자료형을 구현하는 `Coin` 클래스,

`Coin` 배열을 담는 `Bag` 을 구현하는 `ArrayBag` 클래스로 구성된다.

c. 함수 설명서

■ `ArrayBag`

- `public ArrayBag(int givenCapacity)` : 가방의 크기를 설정. 인자를 넣지 않을 시, 100 으로 설정 (생성자)

- `public int size()` : 가방 크기 반환 (Getter)
- `public boolean isEmpty()` : 가방 비어있는지 여부 반환
- `public boolean isFull()` : 가방이 다 찼는지 여부 반환
- `public E elementAt(int order_p)` : 인자로 받은 인덱스에 있는 원소 반환 (Getter)
- `public boolean doesContain(E element_p)` : 인자로 받은 원소와 같은 값의 원소가 있는지 여부 반환
- `public int frequencyOf(E element_p)` : 인자로 받은 원소와 같은 값의 원소가 몇 개 있는지 반환
- `public boolean add(E element_p)` : 인자로 받은 원소를 가방에 추가
- `public boolean remove(E element_p)` : 인자로 받은 원소와 같은 값을 갖는 원소 1 개 제거
- `public void clear()` : 가방 비움
-

■ ApplicationController

- `public void run()` : 가방 프로그램 작동

■ AppView

- `public int inputInt()` : 가방에 들어갈 총 코인 개수 입력받아 반환
- `public int inputMenuNumber()` : 메뉴 번호 입력받아 반환
- `public int inputCoinValue()` : 코인 액수 입력받아 반환
- `public int inputCapacityOfCoinBag()` : 가방에 들어갈 최대 코인 개수 입력받아 반환
- `public void output(String string_p)` : 문자열 출력
- `public void outputLine(String string_p)` : 문자열 줄바꿔서 출력

■ Coin

- `public Coin(int givenValue)` : 코인에 값 설정. (생성자)
- `public int value()` : 코인 값 반환 (Getter)
- `public void setValue(int newValue)` : 코인 값 설정 (Setter)
- `public boolean equals(Object coin_p)` : 코인 값 비교 후 같은 지 반환

d. 종합 설명서

이번 과제는 Bag 자료 구조를 설계하는 것으로, 이 프로그램은 java 의 배열을 이용하여 비순차적이며 중복이 가능한 Bag 구조의 특성을 구현한다.

먼저 가방의 크기를 입력받으며, 이후엔 실행 메뉴를 선택하여 코인의 추가, 삭제, 존재 여부 확인, 특정 코인의 개수 출력, 종료를 할 수 있다.

9 를 입력받으면, 가방에 대한 정보를 출력한 후 종료한다.

2. 프로그램 장단점 분석

장점 : 실제 세계의 주머니처럼 직관적인 정보의 확인이 가능하다.

단점 : 숫자를 입력해야 할 때 정수가 아닌 문자나 실수 등을 입력하면 InputMismatchException 이 발생한다.

3. 실행 결과 분석

a. 입력과 출력

<<< 동전 가방 프로그램을 시작합니다. >>>

가방에 들어갈 코인의 최대 개수 입력 : 20

수행하려는 메뉴 번호 입력

(add : 1, remove : 2, search : 3, frequency : 4, exit : 9) : 1

코인 액수 입력 : 10

주어진 값의 동전을 가방에 성공적으로 넣었습니다.

수행하려는 메뉴 번호 입력

(add : 1, remove : 2, search : 3, frequency : 4, exit : 9) : 2

코인 액수 입력 : 10

주어진 값을 갖는 동전 하나가 가방에서 정상적으로 삭제되었습니다.

수행하려는 메뉴 번호 입력

(add : 1, remove : 2, search : 3, frequency : 4, exit : 9) : 1

코인 액수 입력 : 100

주어진 값의 동전을 가방에 성공적으로 넣었습니다.

수행하려는 메뉴 번호 입력

(add : 1, remove : 2, search : 3, frequency : 4, exit : 9) : 1

코인 액수 입력 : 500

주어진 값의 동전을 가방에 성공적으로 넣었습니다.

수행하려는 메뉴 번호 입력

(add : 1, remove : 2, search : 3, frequency : 4, exit : 9) : 3

코인 액수 입력 : 100

주어진 값을 갖는 동전이 가방 안에 존재합니다.

수행하려는 메뉴 번호 입력

(add : 1, remove : 2, search : 3, frequency : 4, exit : 9) : 4

코인 액수 입력 : 500

주어진 값을 갖는 동전의 개수는 1 개 입니다.

수행하려는 메뉴 번호 입력

(add : 1, remove : 2, search : 3, frequency : 4, exit : 9) : 123123123

선택된 메뉴 번호 123123123는 잘못된 번호입니다.

수행하려는 메뉴 번호 입력

(add : 1, remove : 2, search : 3, frequency : 4, exit : 9) : 9

총 코인의 개수 : 2

가장 큰 코인 금액 : 500

전체 금액 : 600

<<< 동전 가방 프로그램을 종료합니다. >>>

b. 결과분석

10 코인을 하나 넣었다가 뺐고, 100 코인과 500 코인을 하나씩 넣었다.

3 번(존재 여부 확인) 메뉴에서 100 코인이 존재한다고 하였고,

4 번(빈도수 확인) 메뉴에서 500 코인이 1 개 존재한다고 하였으며

9 번(종료) 의 결과에서 600 원으로 나왔으므로 맞는 결과가 나왔다고 할 수 있다.

4. 소스코드

AppController.java

```
public class AppController {

    // 상수
    private static final int MENU_ADD = 1;
    private static final int MENU_REMOVE = 2;
    private static final int MENU_SEARCH = 3;
    private static final int MENU_FREQUENCY = 4;
    private static final int MENU_EXIT = 9;

    // 인스턴스 변수
    private AppView _appView;
    private ArrayBag<Coin> _coinBag;

    // 생성자
    public AppController() {
        this._appView = new AppView();
    }

    // Coin 추가
    private void addCoin() {
        int coinValue = this._appView.inputCoinValue();
        if(this._coinBag.add(new Coin(coinValue))) {
            this._appView.outputLine("주어진 값의 동전을 가방에 성공적으로  
넣었습니다.");
        }
        else {
            this._appView.outputLine("주어진 값의 동전을 가방에 넣는데  
실패하였습니다.");
        }
    }

    // Coin 제거
    private void removeCoin() {
        int coinValue = this._appView.inputCoinValue();

        if(!this._coinBag.remove(new Coin(coinValue))) {
            this._appView.outputLine("주어진 값을 갖는 동전은 가방 안에 존재하지  
않습니다.");
        }
        else {
            this._appView.outputLine("주어진 값을 갖는 동전 하나가 가방에서 정상적으로  
삭제되었습니다.");
        }
    }

    // Coin 존재 유무 확인
    private void searchForCoin() {
```

```

        int coinValue = this._appView.inputCoinValue();
        if(this._coinBag.contains(new Coin(coinValue))) {
            this._appView.outputLine("주어진 값을 갖는 동전이 가방 안에
존재합니다.");
        }
        else {
            this._appView.outputLine("주어진 값을 갖는 동전은 가방 안에 존재하지
않습니다.");
        }
    }

    // Coin 개수 출력
    private void frequencyOfCoin() {
        int coinValue = this._appView.inputCoinValue();
        int frequency = this._coinBag.frequencyOf(new Coin(coinValue));
        this._appView.outputLine("주어진 값을 갖는 동전의 개수는 " + frequency + " 개
입니다.");
    }

    // 번호 잘못 입력 시 오류 출력
    private void undefinedMenuNumber(int menuNumber_p) {
        this._appView.outputLine("선택된 메뉴 번호 " + menuNumber_p + "는 잘못된
번호입니다.");
    }

    // 전체 Coin의 값 합계 반환
    private int sumOfCoinValues() {
        int sum = 0;
        for(int index = 0; index < this._coinBag.size(); index++) {
            sum += this._coinBag.elementAt(index).value();
        }
        return sum;
    }

    // 가장 큰 Coin의 값 반환
    private int maxCoinValue() {
        int maxCoinValue = 0;
        for(int index = 0; index < this._coinBag.size(); index++) {
            if(maxCoinValue < this._coinBag.elementAt(index).value()) {
                maxCoinValue = this._coinBag.elementAt(index).value();
            }
        }
        return maxCoinValue;
    }

    // 가방 내용 정보 출력
    private void showStatistics() {
        this._appView.outputLine("");
        this._appView.outputLine("총 코인의 개수 : " + this._coinBag.size());
        this._appView.outputLine("가장 큰 코인 금액 : " + this.maxCoinValue());
        this._appView.outputLine("전체 금액 : " + this.sumOfCoinValues());
    }

```

```

// 시작
public void run() {

    // 시작 메시지
    this._appView.outputLine("<<< 동전 가방 프로그램을 시작합니다. >>>");

    // 가방에 넣을 동전 개수 입력
    int capacityOfCoinBag = this._appView.inputCapacityOfCoinBag();
    this._coinBag = new ArrayBag<Coin>(capacityOfCoinBag);

    // 메뉴 번호 입력
    int menuNumber = this._appView.inputMenuNumber();
    while(menuNumber != MENU_EXIT) {

        switch(menuNumber) {

            case MENU_ADD:
                this.addCoin();
                break;

            case MENU_REMOVE:
                this.removeCoin();
                break;

            case MENU_SEARCH:
                this.searchForCoin();
                break;

            case MENU_FREQUENCY:
                this.frequencyOfCoin();
                break;

            default:
                this.undefinedMenuNumber(menuNumber);
        }

        // 새로운 메뉴 번호 입력
        menuNumber = this._appView.inputMenuNumber();
    }

    // 가방 안의 동전 정보 출력
    this.showStatistics();

    // 종료 메시지
    this._appView.outputLine("<<< 동전 가방 프로그램을 종료합니다. >>>");
}
}

```

AppView.java

```

// 입출력

import java.util.Scanner;

public class AppView {

    // 인스턴스 변수

```

```

private Scanner _scanner;

// 생성자
public AppView() {
    this._scanner = new Scanner(System.in);
}

// 입력 메소드

// 코인 개수 입력받아 반환
public int inputInt() {

    int coinAmount;
    this.outputLine("");
    this.output("가방에 들어갈 총 코인 개수 입력 : ");
    coinAmount = _scanner.nextInt();

    return coinAmount;
}

// 메뉴 번호 입력받아 반환
public int inputMenuNumber() {

    int menuNumber;
    this.outputLine("");
    this.outputLine("수행하려는 메뉴 번호 입력");
    this.output("(add : 1, remove : 2, search : 3, frequency : 4, exit : 9) : ");
    menuNumber = _scanner.nextInt();

    return menuNumber;
}

// 코인 액수 입력받아 반환
public int inputCoinValue() {

    int coinValue;
    this.output("코인 액수 입력 : ");
    coinValue = _scanner.nextInt();

    return coinValue;
}

// 가방에 들어갈 최대 코인 개수 입력받아 반환
public int inputCapacityOfCoinBag() {

    int capacityOfCoinBag;
    this.output("가방에 들어갈 코인의 최대 개수 입력 : ");
    capacityOfCoinBag = _scanner.nextInt();

    return capacityOfCoinBag;
}

// 출력 메소드

```



```

// 출력
public void output(String string_p) {
    System.out.print(string_p);
}

// 줄바꿈 출력
public void outputLine(String string_p) {
    System.out.println(string_p);
}
}

```

ArrayBag.java

```

// 가방에 대한 정보

public class ArrayBag<E> {

    // 상수
    private static final int DEFAULT_CAPACITY = 100;

    // 인스턴스 변수

    // 가방 최대 용량
    private int _capacity;

    // 현재 가방 용량
    private int _size;

    // 가방은 E의 배열로 구성됨
    private E[] _elements;

    // 생성자
    public ArrayBag() {
        this._capacity = ArrayBag.DEFAULT_CAPACITY;
        this._elements = (E[])new Object[this._capacity];
        this._size = 0;
    }

    public ArrayBag(int givenCapacity) {
        this._capacity = givenCapacity;
        this._elements = (E[])new Object[this._capacity];
        this._size = 0;
    }

    // 현재 크기 반환 : getter
    public int size() {
        return this._size;
    }

    // 가방 비어있는지 여부 반환
    public boolean isEmpty() {
        return (this._size == 0);
    }
}

```

```

// 가방 다 찼는지 여부 반환
public boolean isFull() {
    return (this._size == _capacity);
}

// 주어진 order에 있는 원소 반환
public E elementAt(int order_p) {
    if((0 <= order_p) && (order_p < this.size())) {
        return this._elements[order_p];
    }
    else {
        return null;
    }
}

// 주어진 element과 같은 값의 element이 있는지 여부 반환
public boolean doesContain(E element_p) {

    boolean found = false;

    for(int index = 0; index < this._size; index++) {
        if(element_p.equals(this._elements[index])) {
            found = true;
            break;
        }
    }
    return found;
}

// 주어진 element과 같은 값의 element이 몇 개 있는지 반환
public int frequencyOf(E element_p) {

    int frequencyCount = 0;

    for(int index = 0; index < this._size; index++) {
        if(element_p.equals(this._elements[index])) {
            frequencyCount++;
        }
    }
    return frequencyCount;
}

// 가방에 element 추가
public boolean add(E element_p) {

    if(this.isFull()) {
        return false;
    }
    else {
        this._elements[this._size] = element_p;
        this._size++;
        return true;
    }
}

// 가방에서 주어진 값의 element 삭제
public boolean remove(E element_p) {

    boolean found = false;

```

```

        if(this.isEmpty()) {
            return found;
        }

        // 삭제 시
        else {

            // 가방에 element 1개뿐일 때, 그냥 비우기
            if(this._size == 1) {
                found = this._elements[0].equals(element_p);
                this._elements[0] = null;
                this._size--;
            }

            // 가방에 element가 2개 이상일 때
            for(int index = 0; index < this._size - 1 && !found; index++) {

                found = this._elements[index].equals(element_p);

                // 마지막 원소는 순회되지 않으므로 따로 확인
                found = this._elements[this._size - 1].equals(element_p);

                // 같은 것을 찾으면, 그 인덱스부터 배열 요소 번호들을 1씩 앞으로
                // 당기고 마지막 원소는 null로 초기화
                if(found == true) {
                    for(int j = index; j < this._size - 1; j++) {
                        this._elements[j] = this._elements[j + 1];
                    }
                    this._elements[this._size - 1] = null;
                    this._size--;
                }
            }

            return found;
        }
    }

    // 가방 초기화
    public void clear() {
        this._size = 0;
    }
}

```

Coin.java

// 동전에 대한 정보

```
public class Coin {
```

// 인스턴스 변수

// 금액

```
    private int _value;
```

// 생성자

```

    public Coin() {
    }
    public Coin(int givenValue) {
        this._value = givenValue;
    }

    // 코인 금액 반환 : getter
    public int value() {
        return this._value;
    }

    // 코인 금액 설정 : setter
    public void setValue(int newValue) {
        this._value = newValue;
    }

    // 코인 금액 비교
    @Override
    public boolean equals(Object coin_p) {

        // 인자로 받은 값이 Coin형이 아닐 경우
        if(coin_p.getClass() != Coin.class) {
            return false;
        }

        // Coin형을 받았을 경우 금액 비교
        else {
            return (this.value() == ((Coin)coin_p).value());
        }
    }
}

```

DS1_02_201702081_최재범.java

```

// main

public class DS1_02_201702081_최재범 {

    public static void main(String[] args) {
        AppController appController = new AppController();
        appController.run();
    }
}

```